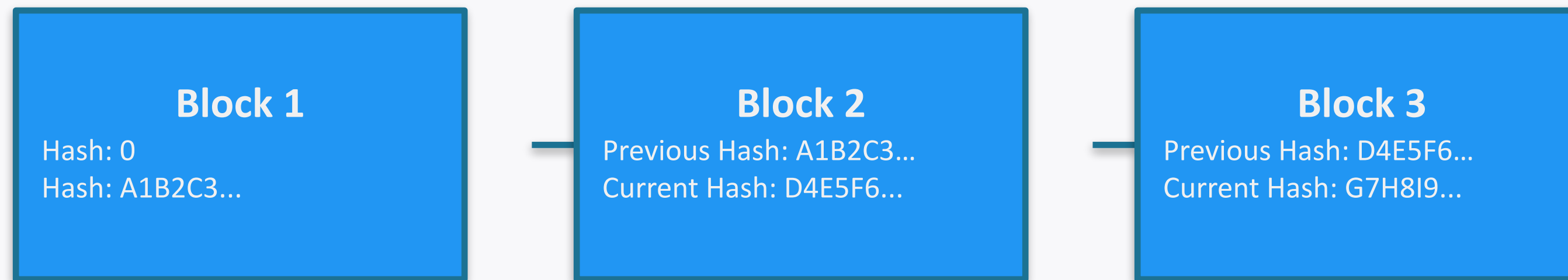


Module 1

Foundations: What Is Ethereum?

What Is a Blockchain?



A blockchain is a decentralized, distributed, and immutable digital ledger that records transactions across a network of computers.

Each block references the previous block's hash, creating an unbreakable chain. Changing any old block would invalidate all blocks that came after it.

Understanding Blockchain Components

Key Terms:

Ledger: Record of balances and data

- A ledger refers to the distributed, immutable, and public record of all transactions, smart contract states, and account balances.

Distributed: Copied across many machines

- Every full node in the Ethereum network keeps a complete, up-to-date copy of the transaction ledger (the blockchain)

Linked Blocks: Prevents tampering

- Refers to the cryptographic mechanism where each new block of transactions contains the unique identifier, known as a **hash**, of the preceding block. This connection forms an unbroken, chronological "chain" of data from the current head of the blockchain all the way back to the Genesis Block.

Each Block Contains:

Transactions: Instructions for state changes

Hash of Previous Block: Creates the 'chain'

Timestamp: When the block was created

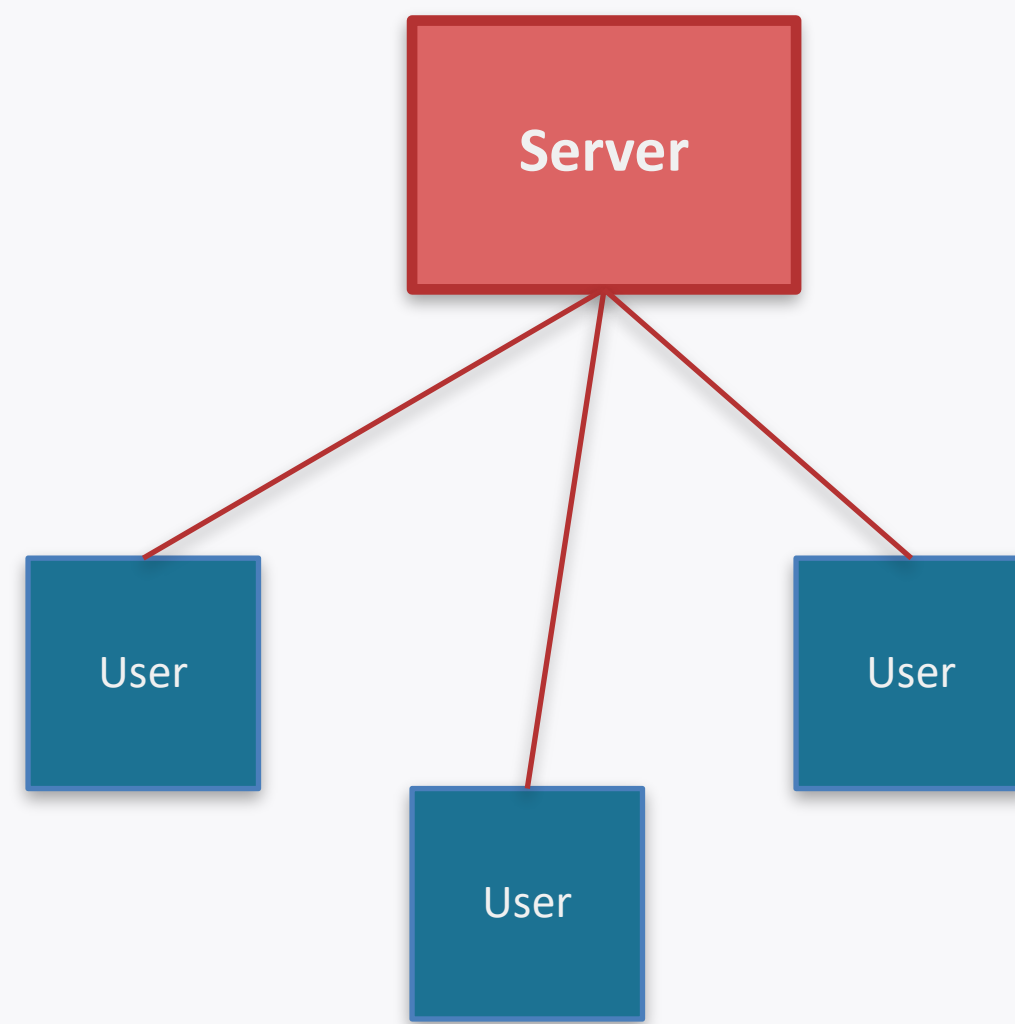
Why the chain is secure:

Changing old data requires recalculating every block after it

The entire network would reject the altered history

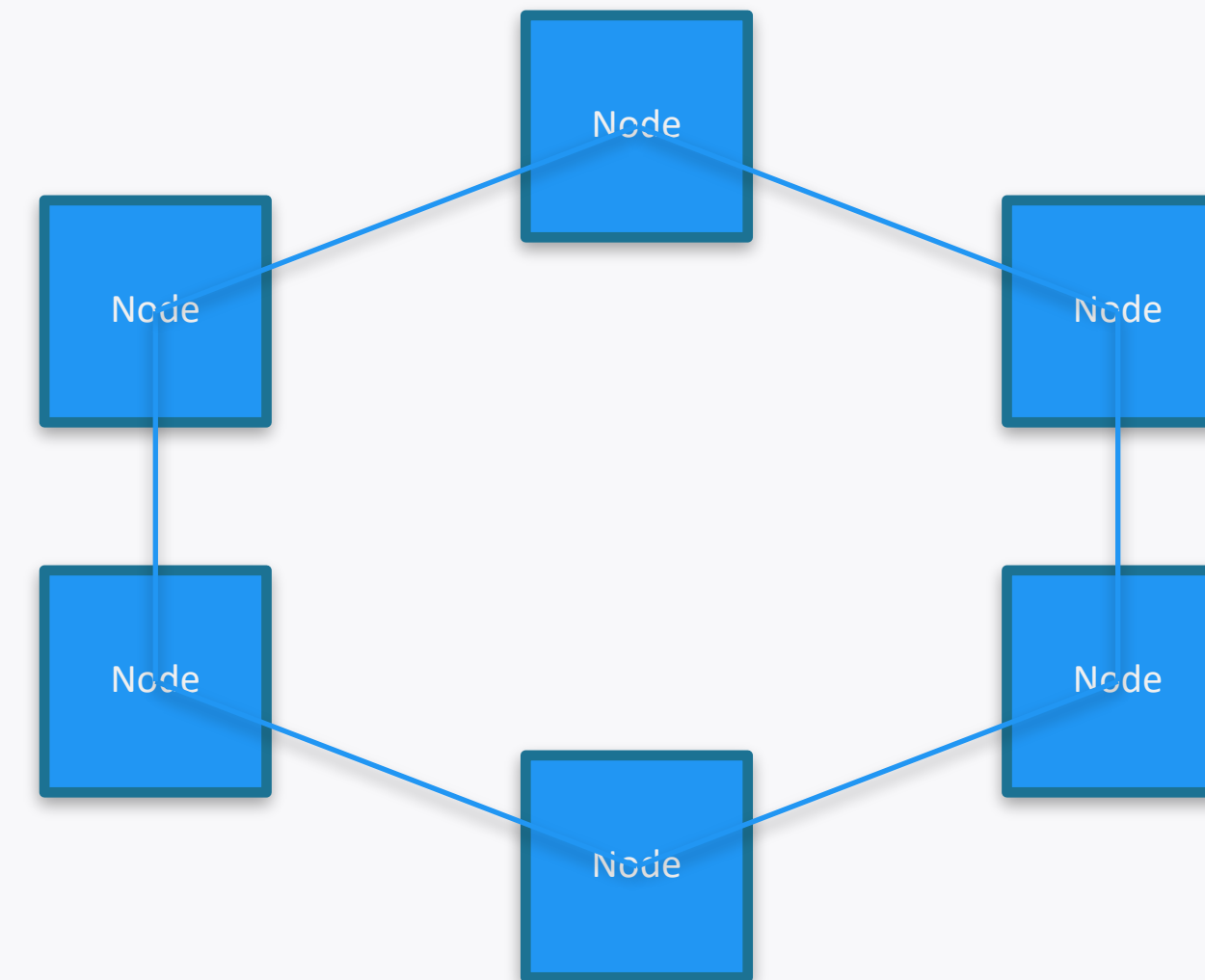
What Is Decentralization?

Traditional (Centralized)



Single point of failure. If server goes down or is hacked, all data is lost!

Ethereum (Decentralized)



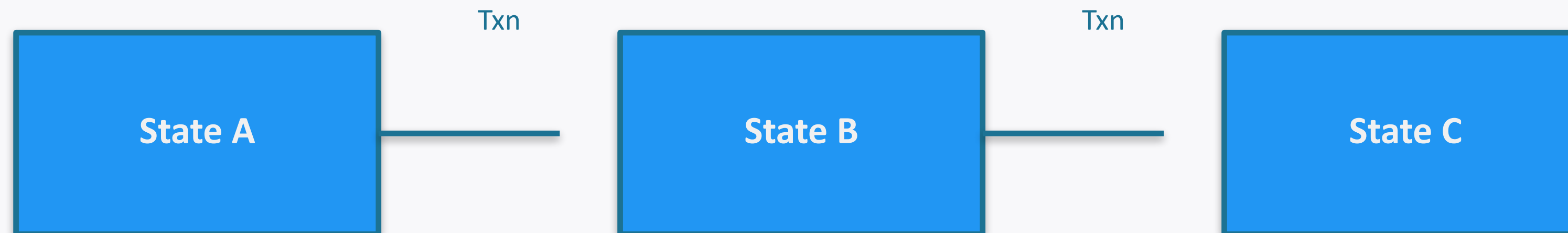
No single point of failure. All nodes store the data. No central authority needed. If one node goes down, the other nodes have a complete copy of the latest state.

Ethereum as a Replicated State Machine

This is where thousands of independent computer nodes across the globe run the same software to execute the same transactions in the same order, resulting in an identical, synchronized, and secure copy of the blockchain's "world state"

Why This Architecture Matters

- It allows Ethereum to host complex, interactive applications (dApps) and smart contracts, not just simple value transfers
- Every node verifies every transaction's execution (re-runs the EVM), ensuring no invalid state transitions occur
- It allows distributed, mutually distrusting participants to agree on a single, shared state without a central authority.



In short, Ethereum does not just store transactions, it stores and updates a shared state according to deterministic transition rules, ensuring that all participants reach the same conclusion regarding the network's status.

The Problem: Trusted Intermediaries

Before Ethereum, you needed a middleman for:

Online payments → PayPal

Lending → Bank

Marketplace → Amazon

The Problem:

Middlemen control and can modify your transactions

You must trust them not to disappear, censor, or fail

They take a cut and control the rules

Ethereum's Solution:

Code runs exactly as written without trusting a company

Cryptographically signed transactions cannot be faked

Global execution engine for programmable agreements

Bitcoin vs Ethereum vs Stellar

Bitcoin

◆ Digital Gold

- Proof of Work
- High energy use
- ~10 min blocks
- Limited programs
- High Barrier to entry as mining is energy intensive

Ethereum

◆ Programmable

- Proof of Stake.
- Energy efficient
- ~12 sec blocks
- Full smart contracts
- High Barrier to entry as running a full node is expensive (32 ETH)
- Liquid Staking protocols solve this removing barrier to entry.

Stellar

● Fast Payments

- Stellar Consensus Protocol
- Very low fees
- ~3-5 sec finality
- Limited programs
- No incentive mechanism for validators doesn't encourage network participation.

Two Types of Ethereum Accounts

Externally Owned Accounts (EOAs):

- Controlled by a private key
- Can initiate transactions
- Represents user wallets
- Users need to keep a secure secret phrase safe in order to secure account. If secret phrase is lost, funds are lost forever.
- Clunky user interface for non technical users
- Eg, Metamask, Rabbi Wallet, Phantom

Contract Accounts:

- Contain smart contract code
- Cannot initiate transactions on their own
- Execute code when called by EOA or another contract
- Contracts can be extended to provide web2 functionality such as account retrieval and sign in with gmail making interacting more familiar with non technical people.
- Eg, Safe.io

Critical Insight:

Contracts need an external trigger

They cannot 'wake themselves up'

Only event-driven execution, never spontaneous

Ethereum Transactions

Definition:

- A cryptographically signed instruction initiated by a user-managed account (EOA) that changes the state of the Ethereum Virtual Machine (EVM). These transactions require a gas fee to execute and can involve sending ETH, transferring tokens, or interacting with smart contracts

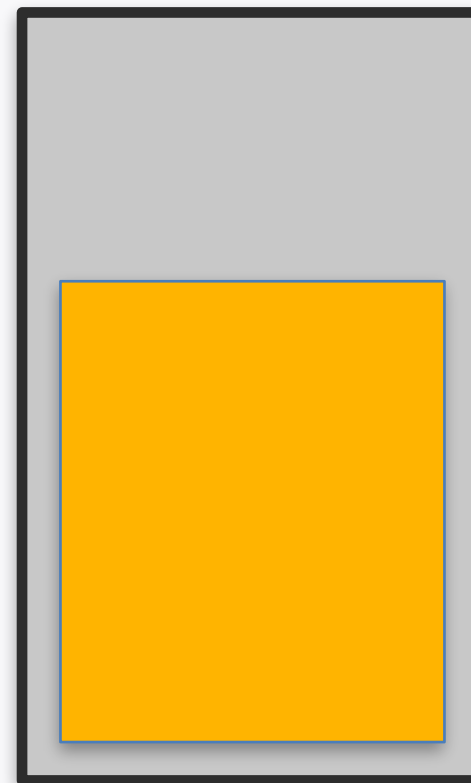
Transaction Properties:

- Signed with a private key (cannot be forged)
- Broadcast to the network
- Modifies Ethereum state
- Consumes computational resources (gas)

Lifecycle:

User signs → Broadcast → Validator includes → EVM executes → State updated

Gas: The Fuel of Ethereum



Gas = Fuel

Gas is the unit that measures the computational effort required to execute operations, such as transactions or smart contract calls, on the Ethereum network. It serves as a transaction fee paid to validators, in ETH, to process, secure, and verify actions, preventing network spam and infinite loops.

Operations and their gas costs:

- Simple transfer: 21,000 gas
- ERC20 Token Transfer: 45,000 - 65,000 gas
- Smart contract call: 21,000+ gas
- Cold read operation: 2200 gas
- Warm read operation 100 gas
- Storage operation: 20,000 gas
- Complex computation: even more

Key insight: Gas prevents spam and infinite loops

How Ethereum Runs: The EVM

Ethereum = World Computer | EVM = CPU

Executes smart contract bytecode

Computes deterministic state transitions

Determinism: Same input + Same computation = Same output

Across all nodes, every time

Enables consensus without central authority

The Process Flow:

Current State + Transaction → EVM Execution → New State

Module 1 Recap: Key Takeaways

- ✓ A blockchain is a distributed, tamper-proof ledger
- ✓ Decentralisation removes the need for trusted middlemen
- ✓ Ethereum solves the problem of trustless execution
- ✓ Ethereum differs from Bitcoin and Stellar in design goals
- ✓ Two account types: EOAs (users) and Contracts (code)
- ✓ Transactions are signed instructions that modify state
- ✓ Gas is the computational cost mechanism
- ✓ The EVM deterministically executes smart contracts

Big Picture:

Ethereum = Decentralised, programmable state machine

No company needed; code runs exactly as written